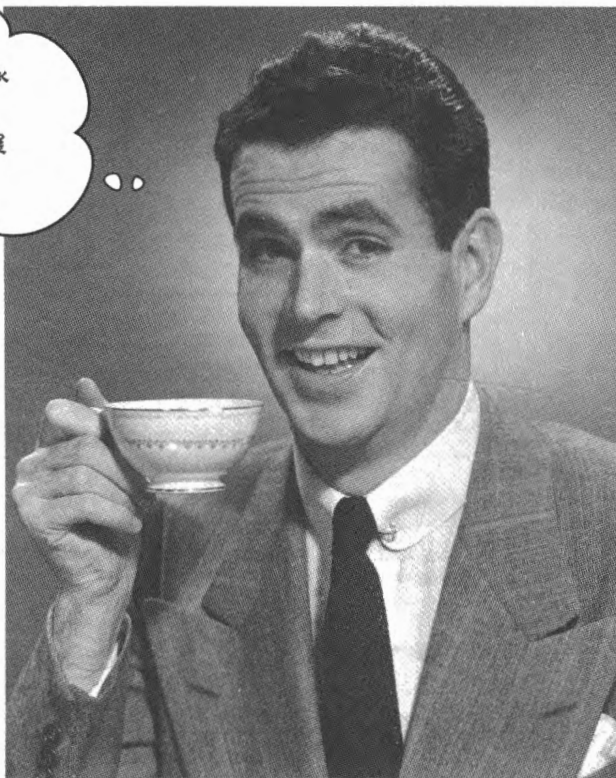


3 装饰者模式

装饰对象

我曾经以为真男人应该用继承处理一切。直到我领教到运行时扩展的威力比编译时扩展的威力要大。看看我现在的样子！



本章可以称为“为爱用继承的家伙打开设计之眼”。

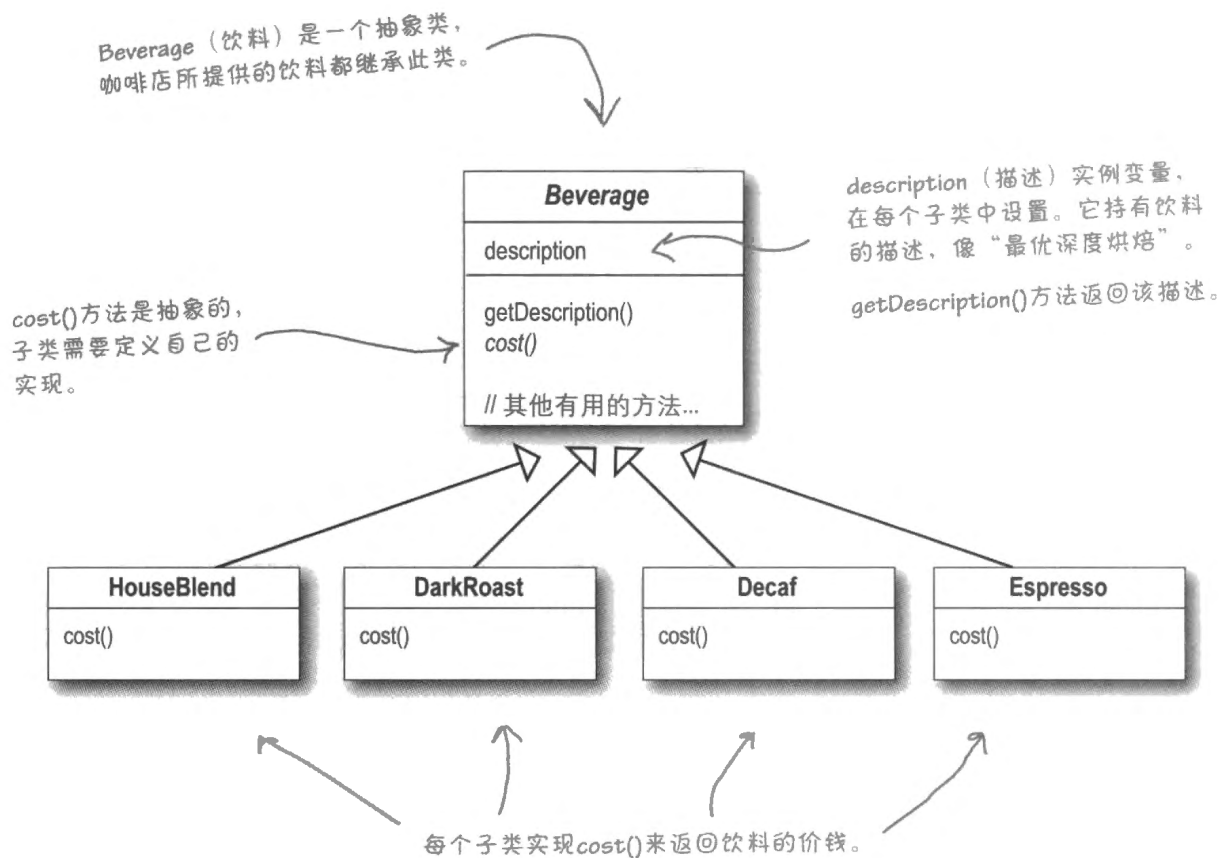
我们即将再度探讨典型的继承滥用问题。你将学到如何在运行时使用对象组合的形式装饰你的类。为什么？一旦知道装饰的技巧，你将能给你的（或别人的）对象一个新的责任，而不需要对底层类的代码做任何改变。

欢迎来到星巴兹咖啡

星巴兹（Starbuzz）咖啡是以扩张速度最快闻名的咖啡连锁店。如果你在街角看到一家它的店，沿街望去，还会看到另一家店。

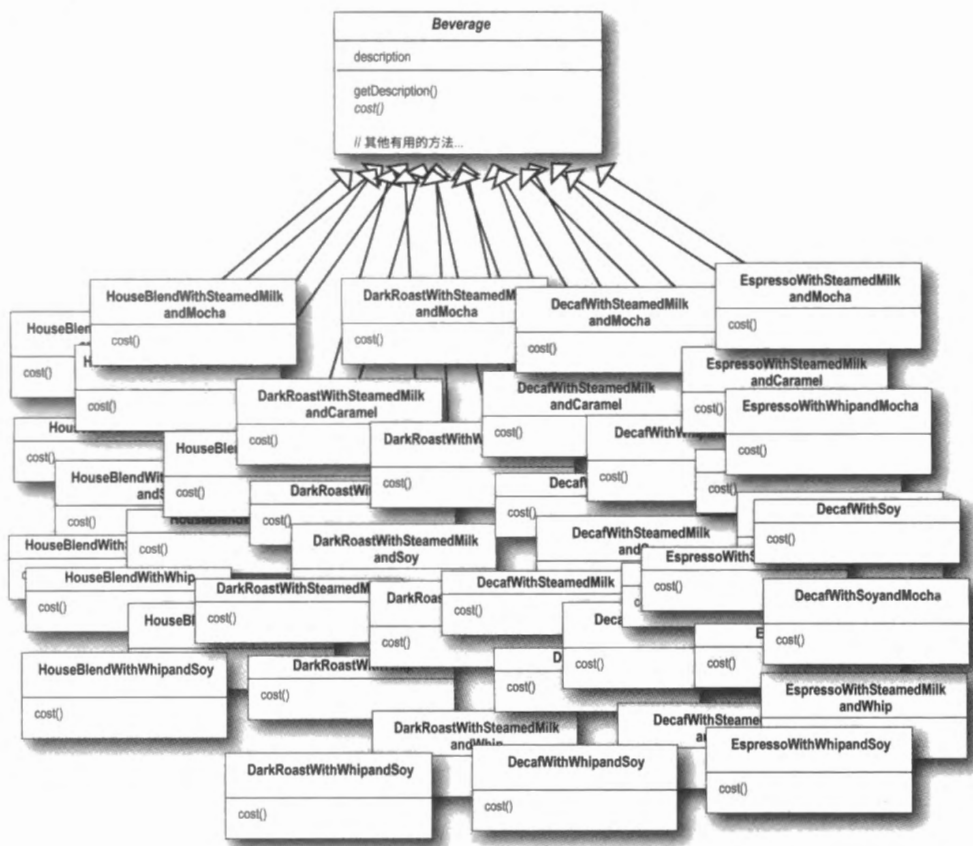
因为扩张实在太快，他们着急更新其下单系统，以匹配他们的饮料供应需求。

当他们第一次进入业务时，类设计像这样……



除了咖啡本身，你也可以要求各种调料，例如：蒸奶（Steamed Milk）、豆奶（Soy）、摩卡（Mocha，也称巧克力风味）或覆盖奶泡。星巴兹会对每个调料收取一点费用，所以他们真的需要把这些构造进订单系统。

这是他们的第一次尝试……



每个cost()方法计算咖啡加上订单上其他调料的价钱。





很明显，星巴兹为自己制造了一个维护噩梦。如果牛奶的价钱上涨，怎么办？新增一种焦糖调料风味时，怎么办？

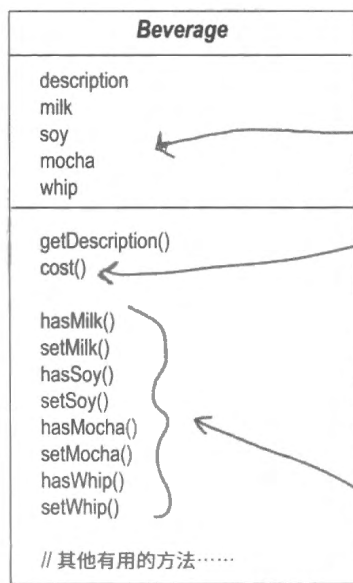
在这个维护问题上拔高来思考，他们违反了我们目前为止提到过的哪个设计原则？

提示：他们违反了两个原则，而且很严重！

蠢。为什么我们需要这么些类？
我们不能只使用实例变量和继承
来跟踪调料吗？



好吧！我们来试试看。先从Beverage基类开始，加上实例变量代表是否加上调料（牛奶、豆奶、摩卡、奶泡……）



每种调料一个新的布尔值。

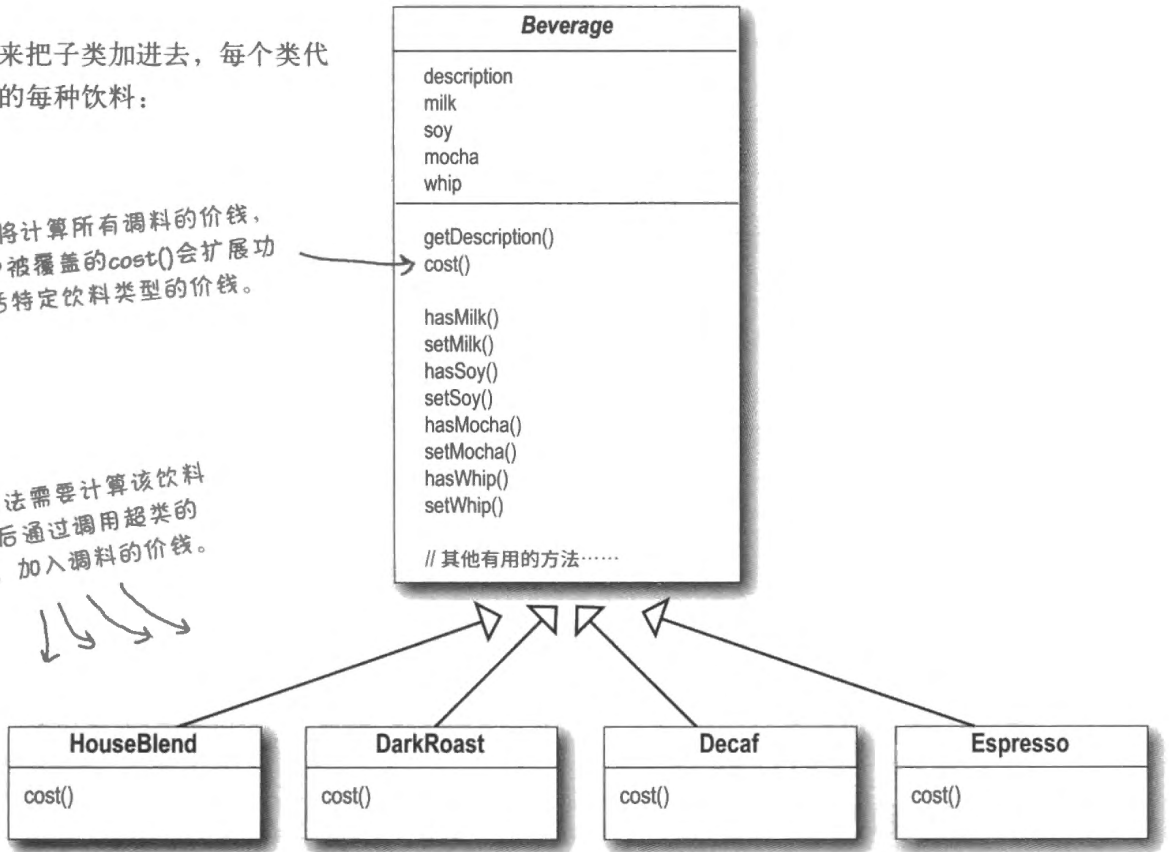
现在我们将要实现Beverage中的cost()，而不是让它保持抽象，这样它可以计算特定饮料实例的调料价钱。子类仍然覆盖cost()，但它们也会调用超类的版本，这样，它们可以计算基本饮料加上添加调料的总价钱。

这些取得和设置调料的布尔值。

现在我们来把子类加进去，每个类代表菜单上的每种饮料：

超类`cost()`将计算所有调料的价钱，而在子类中被覆盖的`cost()`会扩展功能，以包括特定饮料类型的价钱。

每个`cost()`方法需要计算该饮料的价钱，然后通过调用超类的`cost()`实现，加入调料的价钱。



削尖你的铅笔

为以下类编写`cost()`方法（用伪Java就可以）：

```

public class Beverage {
    public double cost() {

    }
}
  
```

```

public class DarkRoast extends Beverage {

    public DarkRoast() {
        description = "Most Excellent Dark Roast";
    }

    public double cost() {

    }
}
  
```



削尖你的铅笔

需求或其他因素可能有什么变化，会影响这个设计？

调料价格改变会迫使我们更改现有代码。

新的调料会迫使我们添加新的方法，并改变超类中的`cost`方法。

可能有新的饮料。对于其中一些饮料（冰茶？），调料可能不适合，然而，`Tea`（茶）子类仍将继承像`hasWhip()`（是否加奶泡）这样的方法。

正如我们在第1章看到的，这是个非常糟糕的想法！

顾客想要双倍摩卡，怎么办？

轮到你了：



大师和徒弟……

大师：距离我们上次见面已经有些时日，你深入冥想过继承了吗？

徒弟：是的，大师。尽管继承威力强大，但是我体会到它并不总是带来最有弹性以及可维护的设计。

大师：啊！是的，你已经有所长进。那么，告诉我，我的徒弟，如果不通过继承，你要怎么达到复用呢？

徒弟：大师，我已经学习到，通过组合和委托，可以在运行时“继承”行为。

大师：好，继续……

徒弟：当我通过子类化继承行为时，行为是编译时静态设置的。另外，所有子类必须继承相同的行为。但是，如果我可以通过组合扩展对象的行为，我就可以在运行时动态地做这件事情。

大师：很好，你正在开始看到组合的威力了。

徒弟：是的，通过这个技巧，我就可以把多个新的责任添加到对象，甚至包括超类的设计师没有想到的责任。我不必碰他们的代码！

大师：关于组合在维护代码方面的效果，你学到了什么？

徒弟：嗯，这正是我要说的。通过动态地组合对象，我可以通过编写新代码添加新的功能，而无须修改已有代码。因为我没有改变已有代码，引进bug或导致意外副作用的机会大幅减少。

大师：非常好。今天的会谈已经够了。希望你能更深入思考这个主题……记住，代码应该像夜晚的莲花一样（对改变）关闭，像早晨的莲花一样（对扩展）开放。

开放-关闭原则

我们在面对最重要的设计原则之一：



设计原则

类应该对扩展开放，但对修改关闭。



请进，我们在开
放中。请自由使用任何
你想要的新行为扩展我们的类。如果你的需
要或需求变了（我们知道它们会的），只需要做
你自己的扩展就可以了。



对不起，我们已经
关闭。

没错，我们花了许多时间把代码弄得正确，没有
bug，因此我们不能让你改变已有代码。它必须
保持对修改关闭。如果你不喜欢，可以找经理谈。

我们的目标是允许类容易扩展以容纳新的行为，而不用修改已有代码。
达成这个目标，有什么好处？这样的设计可以弹性应对改变，有足够弹
性接受新的功能来应对改变的需求。

没有蠢问题

问：对扩展开放和对修改关闭？这听起来是非常矛盾的。设计怎么能两者皆是？

答：这是一个好问题。这乍听之下当然，是矛盾的，毕竟，越难修改的东西，就越难扩展，不是吗？

但是，有某些聪明的OO技巧，允许系统在不改变底层代码的情况下扩展。想想观察者模式（第2章）……通过添加新的观察者，我们可以在任何时候扩展Subject（主题），而不用给Subject添加代码。你还会看到很多其他扩展行为的OO设计技巧。

问：OK，我了解观察者，但是通常怎样将某件东西设计成可以扩展，又对修改关闭？

答：许多模式带给我们经过时间考验的设计。通过提供扩展的方法，保护代码免于被修改。在本章中，你将看到使用装饰者模式来遵循开放-关闭原则的好例子。

问：我怎样能够使得我的设计的每个部分遵循开放-关闭原则？

答：通常，你办不到。让OO设计有弹性，对扩展开放，又不修改已有代码，需要时间和精力。一般来说，我们没有奢侈到在我们设计的每一部分都花费时间（这可能是浪费）。遵循开放-关闭原则，通常会引入新的抽象层次，这会增加代码的复杂度。你要专注于设计中最有可能改变的区域，然后在那里应用开放-关闭原则。

问：我怎么知道哪个区域的变化是更重要的？

答：一部分是靠设计OO系统的经验，还要懂得你工作的领域。看看其他例子，有助于你学会识别你自己设计中的变化区域。

虽然看起来矛盾，但的确有技巧允许代码在不直接修改的情况下扩展。

在选择要扩展的代码区域时，要小心；在每个地方应用开放-关闭原则是浪费和没有必要的，会导致复杂的、难以理解的代码。

结识装饰者模式

OK，我们已经看到，用继承表示饮料和调料，效果不是很好。我们得到类爆炸和僵硬的设计，或者我们添加到基类的功能不适合一些子类。

因此，这是我们的另一种做法：我们将从饮料开始并在运行时用调料“装饰”它。例如，如果顾客要一份带摩卡和奶泡的深度烘焙，那么我们将：

- ❶ 从DarkRoast（深度烘焙）对象开始。
- ❷ 用一个Mocha（摩卡）对象装饰它。
- ❸ 用一个 Whip对象装饰它。
- ❹ 调用cost()方法，依靠委托来累加调料价钱。

OK，但你如何“装饰”一个对象，委托是怎么来的？提示：把装饰者对象想成“包装者”。我们来看看这如何运作……

OK，受够了这帮“面向对象设计俱乐部”的家伙。快来解决真正的问题吧！还记得我们吗？星巴克咖啡？你认为使用这些设计原则中的一些，确实会帮到我们吗？



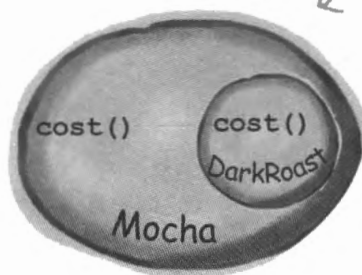
用装饰者构造饮料订单

① 我们从DarkRoast对象开始。



记住，DarkRoast继承自Beverage，有一个cost()方法，该方法计算饮料的价钱。

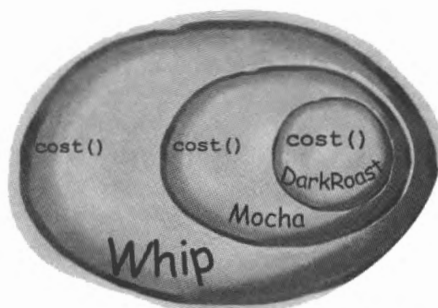
② 顾客要摩卡，所以我们创建一个Mocha对象，并用它包住DarkRoast。



Mocha对象是一个装饰者，它的类型“反映”了它所装饰的对象——本例中，就是Beverage。（“反映”指的是类型相同。）

因此，Mocha也有一个cost()方法，通过多态，也可以把任何被Mocha包裹的Beverage当成是Beverage。（因为Mocha是Beverage的子类型）。

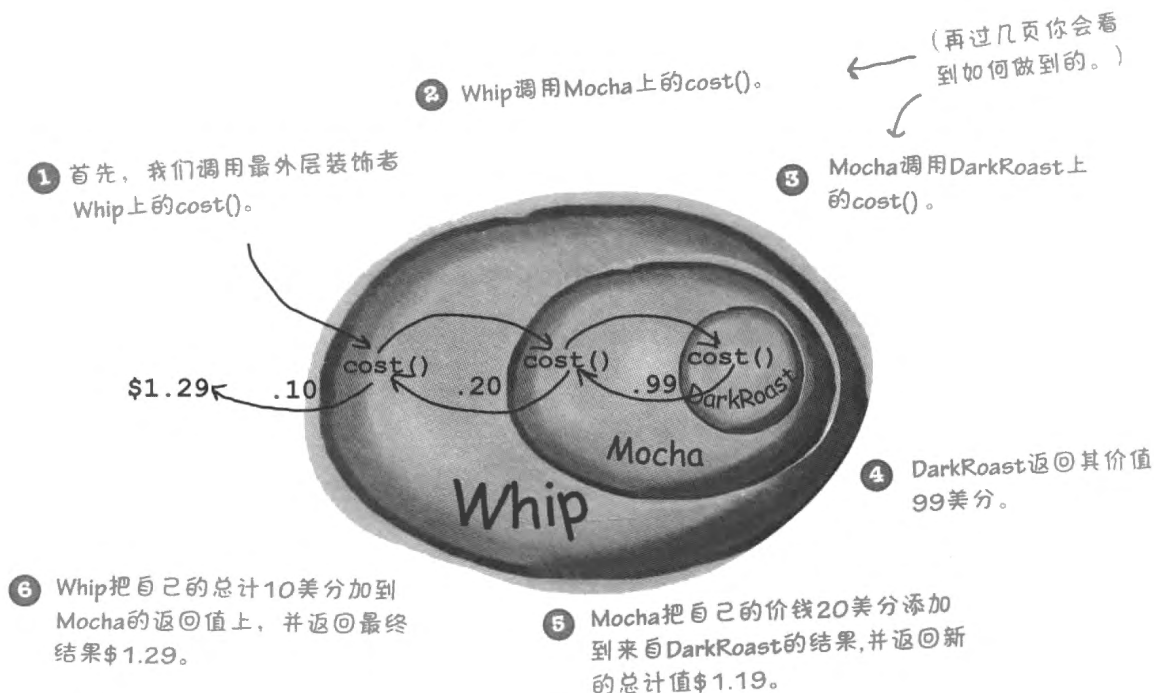
③ 顾客也要奶泡，因此我们创建一个Whip装饰者，并用它包住Mocha。



Whip是一个装饰者，所以它也反映了DarkRoast的类型，并包含一个cost()方法。

因此，被Mocha和Whip包起来的DarkRoast对象，仍然是一个Beverage，我们可以用它做DarkRoast可以做的任何事情，包括调用其cost()方法。

- ④ 现在是时候为顾客计算价钱了。通过调用最外层装饰者Whip上的cost()可以做到。Whip把计算价钱委托给它装饰的对象，等等。我们来看这如何运作：



OK，这是目前为止我们所知道的关于装饰者的一切……

- 装饰者有着和所装饰对象同样的超类型。
- 你可以用一个或多个装饰者包裹一个对象。
- 鉴于装饰者有着和所装饰对象同样的超类型，在需要原始对象的场合，我们可以传递一个被装饰的对象。
- 装饰者在委托给所装饰对象之前或（与）之后添加自己的行为，来做剩下的工作。
- 对象可以在任何时候被装饰，因此我们可以在运行时用任意数量的装饰者动态地装饰对象，只要我们乐意。

关键点！

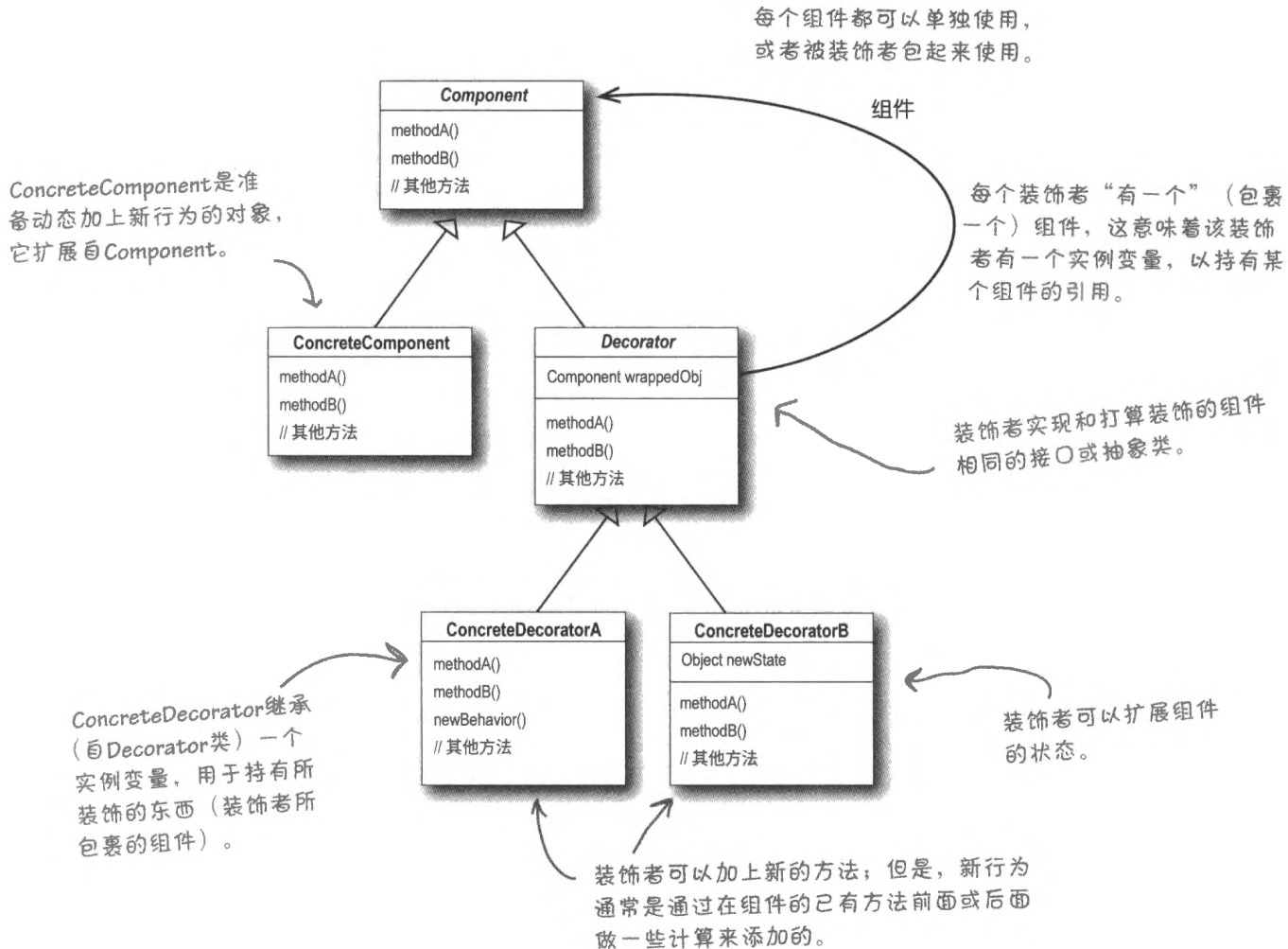
现在，我们来看看装饰者模式的定义，并写一些代码，了解它的所有运作真相。

装饰者模式定义

我们首先来看一看装饰者模式的描述：

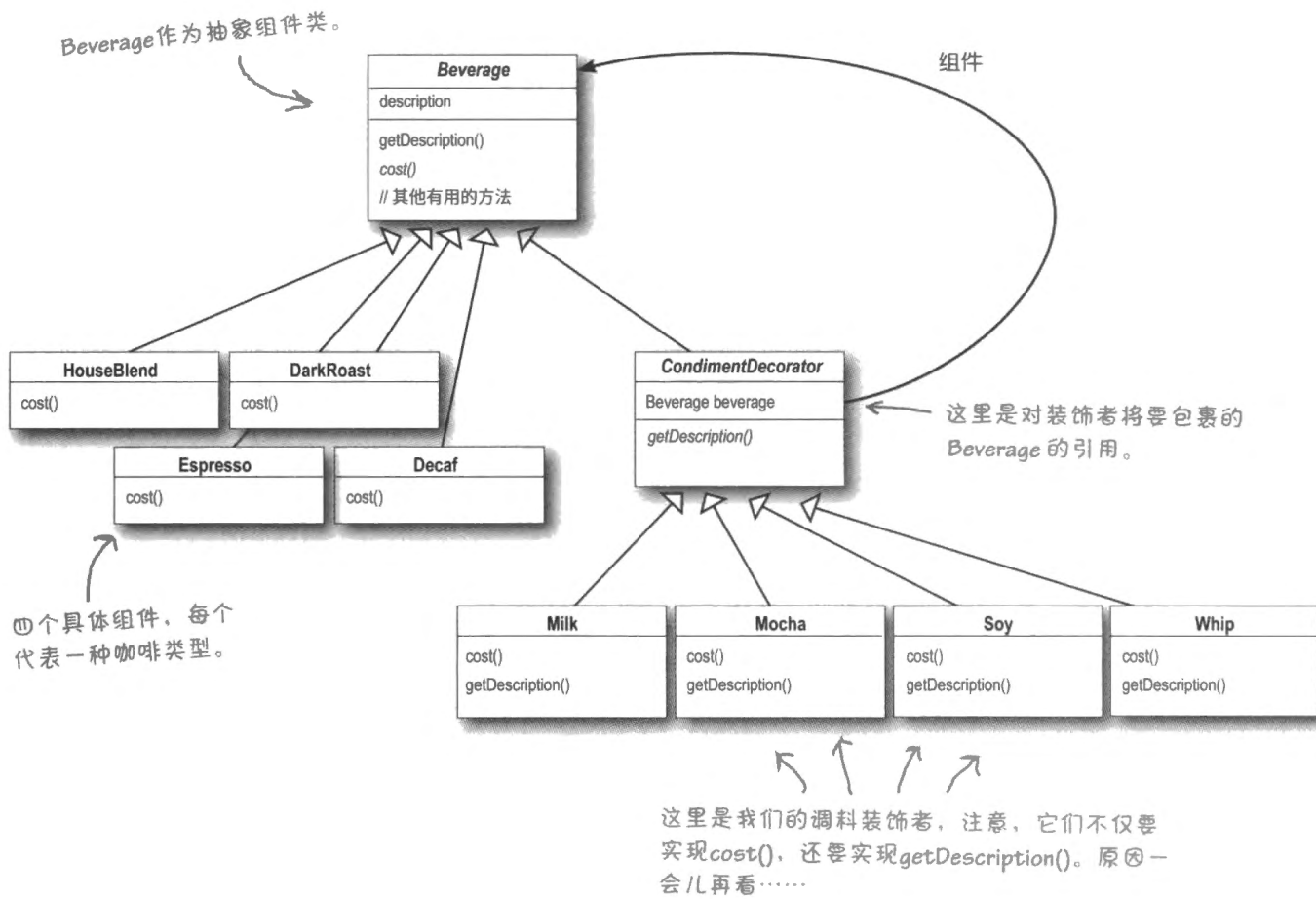
装饰者模式动态地将额外责任附加到对象上。
对于扩展功能，装饰者提供子类化之外的弹性替代方案。

虽然以上描述了装饰者模式的“角色”，但没有告诉我们太多关于应用该模式到我们自己的实现的信息。我们看看类图，可能会更有启发一些（在下一页，我们把同样的结构应用在饮料问题上）。



装饰我们的饮料

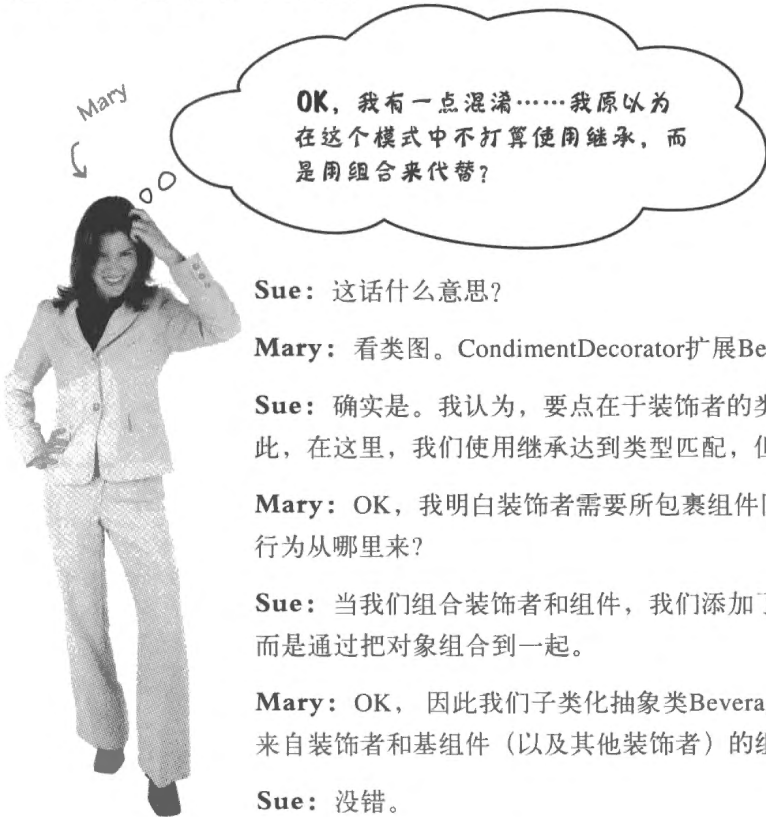
我们使用装饰者模式来重做星巴兹饮料……



往下看之前，想想如何实现咖啡和调料的cost()方法。也思考一下如何实现调料的getDescription()方法。

办公室隔间对话

继承 vs. 组合的一些混淆



Sue: 这话什么意思?

Mary: 看类图。CondimentDecorator扩展Beverage类。这是继承，对吧?

Sue: 确实是。我认为，要点在于装饰者的类型和打算装饰的对象相同，这是至关重要的。因此，在这里，我们使用继承达到类型匹配，但不使用继承来获得行为。

Mary: OK，我明白装饰者需要所包裹组件同样的“接口”，因为它们需要充当组件。但是，行为从哪里来?

Sue: 当我们组合装饰者和组件，我们添加了新的行为。我们不是通过继承超类获得新行为，而是通过把对象组合到一起。

Mary: OK，因此我们子类化抽象类Beverage是为了有正确的类型，而不是继承其行为。行为来自装饰者和基组件（以及其他装饰者）的组合。

Sue: 没错。

Mary: 噢，我懂了！而且因为使用对象组合，在如何混合与匹配调料和饮料方面，我们有了很大的弹性，非常灵活。

Sue: 是的，如果我们依靠继承，那么我们的行为只能在编译时静态地决定。换句话说，我们只得到超类给我们的行为或者覆盖它们。使用组合，我们可以用我们喜欢的方式在运行时混合与匹配装饰者。

Mary: 我懂了一在任何时候，我们可以实现新的装饰者来添加新的行为。如果我们依赖于继承，任何时候需要新行为，我们都不得不进入已有代码并修改它。

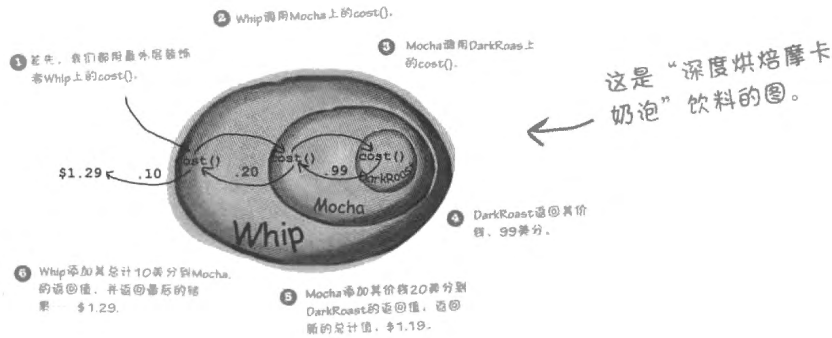
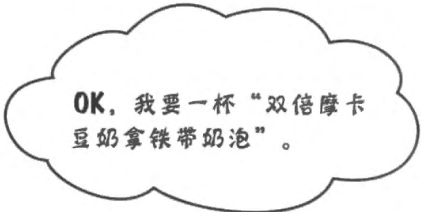
Sue: 正是如此。

Mary: 我还剩下一个问题：如果我们需要继承的是组件的类型，我们怎么不用一个接口代替Beverage抽象类?

Sue: 嗯，记住，我们得到代码时，星巴兹已经有了一个抽象Beverage类。通常，装饰者模式确实指定一个抽象组件，但在Java中，显然，我们可以使用一个接口。但我们总是尝试避免修改已有代码，因此，如果抽象类运作得好好的，就不要去“修正”它了。

新咖啡师训练

为订单“双倍摩卡豆奶拿铁带奶泡”画一张图，使用菜单得到正确的价格，用和之前一样的格式来画图（翻回几页前）：



削尖你的铅笔

把图画在这里。

星巴兹咖啡	
咖啡	
家常综合	.89
深度烘焙	.99
低咖啡因	1.05
浓缩	1.99
调料	
热奶	.10
摩卡	.20
豆奶	.15
奶泡	.10



编写星巴兹代码

是时候把这个设计转成一些真正的代码了。

我们从Beverage类开始，不需要改变星巴兹的原始设计。

我们来看看：



```
public abstract class Beverage {
    String description = "Unknown Beverage";

    public String getDescription() {
        return description;
    }

    public abstract double cost();
}
```

Beverage是一个抽象类，有两个方法，getDescription()和cost()。

getDescription已经实现，但需要在子类中实现cost()。

Beverage足够简单。我们也来为调料实现抽象类（装饰者）：

```
public abstract class CondimentDecorator extends Beverage {
    Beverage beverage;
    public abstract String getDescription();
}
```

首先，我们需要和Beverage互换，因此我们扩展Beverage类。

这里是每个装饰者将要包裹的Beverage。注意，我们使用Beverage超类型来引用到Beverage，因此，装饰者可以包裹任何饮料。

我们也打算要求所有调料装饰者都重新实现getDescription()方法。同样，原因一会再说……

编写饮料代码

现在，我们有了基类，我们来实现一些饮料。我们先从浓缩咖啡（Espresso）开始。记住，我们需要为特定饮料设置一个描述，还要实现cost()方法。

```
public class Espresso extends Beverage {
```

```
    public Espresso() {
```

```
        description = "Espresso";
```

```
    }
```

```
    public double cost() {
```

```
        return 1.99;
```

```
    }
```

```
}
```

首先，我们扩展Beverage类，
因为这是一个饮料。

我们把设置描述放在类的构造器。
记住，description实例变量继承自
Beverage。

最后，我们需要计算Espresso的价钱。这个类中，我们不需要管添
加调料价钱的事情，只需要返回Espresso的价钱\$1.99。

```
public class HouseBlend extends Beverage {
```

```
    public HouseBlend() {
```

```
        description = "House Blend Coffee";
```

```
    }
```

```
    public double cost() {
```

```
        return .89;
```

```
    }
```

```
}
```

OK，这是另一个Beverage，我们要做的是
设置恰当的描述“House Blend Coffee”，
然后返回正确的价钱89¢。

你可以创建另外两种饮料（DarkRoast和Decaf），
做法完全一样。

星巴兹咖啡

咖啡	.89
家常综合	.99
深度烘焙	1.05
低咖啡因	1.99
浓缩	

调料	.10
热奶	.20
摩卡	.15
豆奶	.10
奶泡	

编写调料代码

如果你回头看装饰者模式类图，你会看到现在我们编写了抽象组件（Beverage），还有具体组件（HouseBlend）以及抽象装饰者（CondimentDecorator）。现在是时候实现具体装饰者了。这是Mocha（摩卡）：

Mocha是一个装饰者，因此我们扩展CondimentDecorator。

记住，CondimentDecorator扩展Beverage。

```
public class Mocha extends CondimentDecorator {
```

```
    public Mocha(Beverage beverage) {
        this.beverage = beverage;
    }
```

```
    public String getDescription() {
        return beverage.getDescription() + ", Mocha";
    }
```

```
    public double cost() {
        return beverage.cost() + .20;
    }
}
```

我们打算用一个Beverage引用来实例化Mocha。

记住，这个类继承Beverage实例变量，以持有正在包裹的饮料。

我们设置这个实例变量为正在包裹的对象。这里，我们传递正在包裹的饮料给装饰者的构造器。

我们希望描述不仅包含饮料（例如“DarkRoast”），也包含装饰饮料的每一样东西（例如“DarkRoast, Mocha”），因此我们首先委托给正在装饰的对象来获取其描述，然后把“， Mocha”附加到描述后面。

现在我们需要计算带摩卡的饮料的价格。首先，我们把调用委托给正在装饰的对象，这样，它可以计算价格；然后，我们把摩卡的价格添加到结果。

在下一页，我们将实际地实例化饮料，并用所有调料（装饰者）包裹它，但首先……



练习

编写并编译另外的Soy和Whip调料的代码。为了完成和测试应用，你需要它们。

提供一些咖啡

恭喜！是时候舒服地坐下来，买一些咖啡，欣赏你用装饰者模式创建的弹性设计了。

这里是一些下订单的测试代码*：

```
public class StarbuzzCoffee {
```

```
    public static void main(String args[]) {
        Beverage beverage = new Espresso();
        System.out.println(beverage.getDescription()
            + " $" + beverage.cost());
```

要一杯浓缩咖啡，不加调料，打印出它的描述和价格。

```
        Beverage beverage2 = new DarkRoast();
```

做一个DarkRoast对象。

```
        beverage2 = new Mocha(beverage2);
```

用一个Mocha包裹它。

```
        beverage2 = new Mocha(beverage2);
```

用第二个Mocha包裹它。

```
        beverage2 = new Whip(beverage2);
```

用一个Whip包裹它。

```
        System.out.println(beverage2.getDescription()
            + " $" + beverage2.cost());
```

```
        Beverage beverage3 = new HouseBlend();
```

```
        beverage3 = new Soy(beverage3);
```

```
        beverage3 = new Mocha(beverage3);
```

```
        beverage3 = new Whip(beverage3);
```

```
        System.out.println(beverage3.getDescription()
            + " $" + beverage3.cost());
```

最后，给我们一份带Soy、Mocha和Whip的HouseBlend。

```
    }
```

```
}
```

* 当我们讲到工厂和生成器模式时，我们会看到更好的创建装饰对象的方式。请注意，生成器模式放在附录。

现在，我们来把这些订单放进去：

```
File Edit Window Help CloudsInMyCoffee
% java StarbuzzCoffee
Espresso $1.99
Dark Roast Coffee, Mocha, Mocha, Whip $1.49
House Blend Coffee, Soy, Mocha, Whip $1.34
%
```

没有蠢问题

问：我有点担心测试特定具体组件（也就是说，HouseBlend的代码），做一些类似打折的事情。一旦我用装饰者包裹HouseBlend，代码就不再工作了。

答：的确是。如果你的代码依赖于具体组件类型，装饰者会破坏这个代码。只要只针对抽象组件类型编码，装饰者的使用会对你的代码保持透明。但是，一旦你开始针对具体组件编码，你要重新思考应用的设计以及装饰者的使用。

问：对于饮料的一些客户来说，会不会容易不使用并非最外层的装饰者？比如说，如果有一个带Mocha、Soy和Whip的DarkRoast，编码时容易引用到Soy而不是Whip，这意味着订单里不包含Whip。

答：你当然可以争辩说，使用装饰者模式，你不得不管理更多的对象，因此，犯下你所说问题的编码错误的可能性会增加。但是，我们通常使用其他模式创建装饰者，像工厂和生成器。一旦我们学到了这些模式，你会看到，带装饰者的具体组件的创建“封装得很好”，不会导致这种问题。

问：装饰者可以知道链条上的其他装饰吗？比如说，我要我的getDescription()方法打印“Whip, Double Mocha”，而不是“Mocha, Whip, Mocha”。这就需要我的最外层装饰者知道它包裹的所有装饰者。

答：装饰者意味着添加行为到所包裹的对象。当你需要窥探装饰者链条的多个层时，就已经超出装饰者真正的意图了。尽管如此，还是有可能做到的。想象一个CondimentPrettyPrint装饰者，它解析最后的描述，并打印“Mocha, Whip, Mocha”为“Whip, Double Mocha”。注意，getDescription()可以返回一个描述的ArrayList，这样更容易编码。



削尖你的铅笔

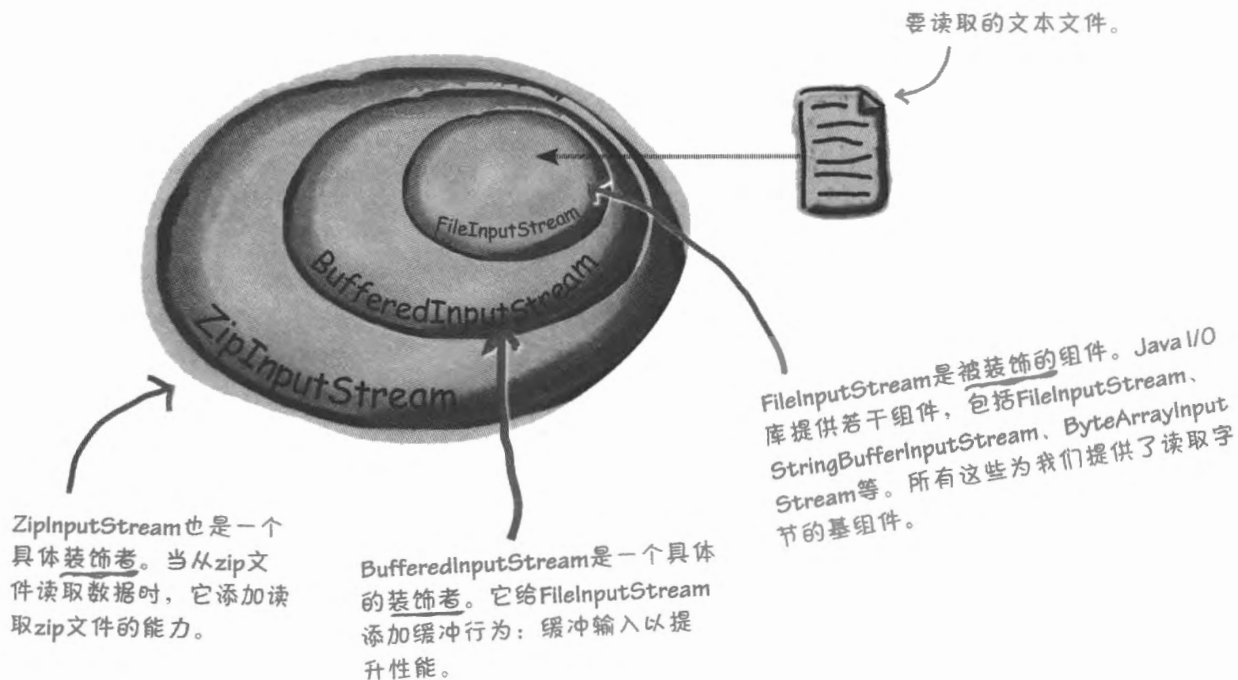
我们在星巴克的朋友在菜单中引进了容量。现在你可以购买小杯（tall）、中杯（grande）、大杯（venti）。星巴克认为这是咖啡行业必备的，因此他们给Beverage类添加了两方法：setSize()和getSize()。他们也希望调料根据容量收费，例如：小、中、大容量的咖啡加上豆奶，分别加收10美分、15美分、20美分。更新后的Beverage类如下。

怎样修改装饰者类来处理这样的需求变更？

```
public abstract class Beverage {
    public enum Size { TALL, GRANDE, VENTI };
    Size size = Size.TALL;
    String description = "Unknown Beverage";
    public String getDescription() {
        return description;
    }
    public void setSize(Size size) {
        this.size = size;
    }
    public Size getSize() {
        return this.size;
    }
    public abstract double cost();
}
```

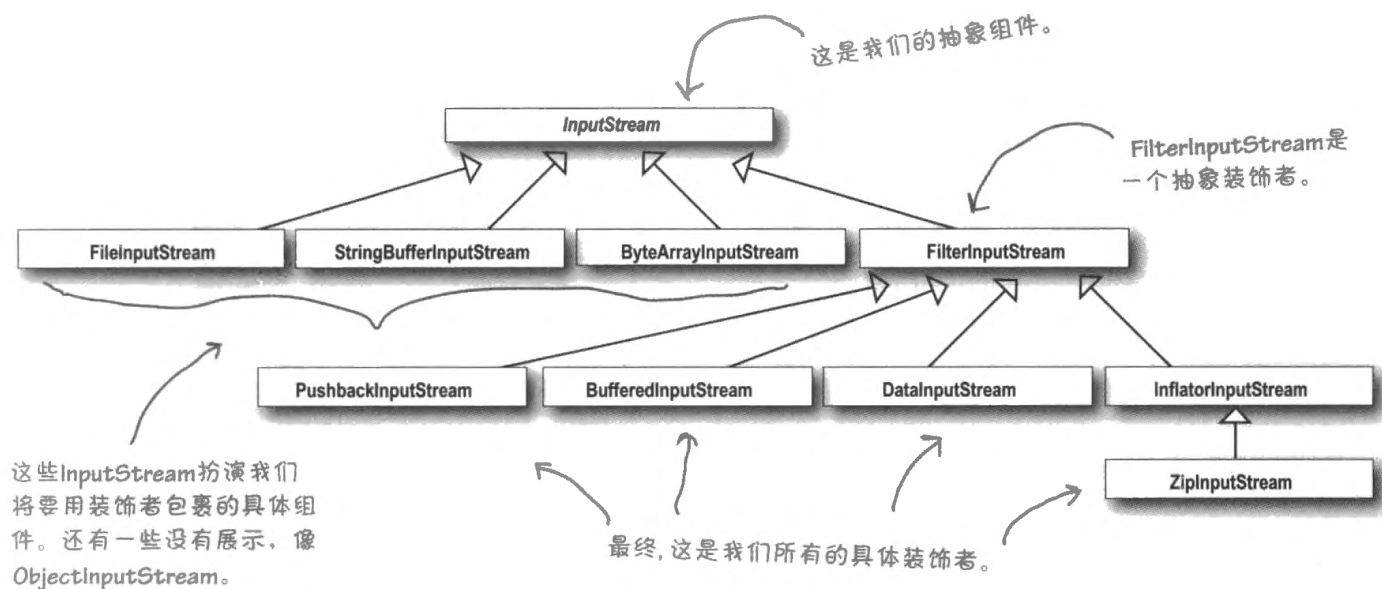
现实世界的装饰者：Java I/O

java.io包内的类太多了，简直是“排山倒海”。不止你一个人在第一次（还有第二次和第三次）看到这API时会发出“哇哦”的一声。但现在，你已经知道了装饰者模式，应该更容易理解I/O类了，因为java.io包很大程度上基于装饰者模式。以下是一个典型的对象集，使用装饰者来添加功能，从文件读取数据。



装饰java.io类

BufferedInputStream和ZipInputStream都扩展自FilterInputStream，而FilterInputStream扩展自InputStream。InputStream扮演抽象装饰者类：



你可以看到，和星巴兹的设计相比没有太多差异。现在，你应该站在了一个好的位置，来观察java.io API文档，并组合不同的输入流装饰者。

你会发现“输出”流的设计方式也是一样的。可能还发现Reader/Writer流（作为基于字符数据的输入输出）和输入流/输出流的类相当类似（虽然有一些小差异和不一致之处，但是相当雷同，因此你应该可以了解这些类）。

Java I/O也引出了装饰者模式的一个缺点：使用这个模式来设计，经常造成大量的小类，数量巨大到给试图使用基于装饰者的API的开发人员带来困扰。但现在你知道装饰者是如何工作的，当你使用其他人的重度装饰的API时，心里就有数了，你可以浏览一遍，看它们的类如何组织，这样，你可以容易地通过包裹来获得想要的行为。

编写你自己的Java I/O装饰者

OK，你知道了装饰者模式，你也看到了I/O类图。你应该准备编写你自己的输入装饰者了。

这题目如何：编写一个装饰者，把输入流中的所有大写字符转成小写。比如，如果读入“I know the Decorator Pattern therefore I RULE!”，装饰者会将它转成“i know the decorator pattern therefore i rule!”。

别忘了导入java.io……
(此处未显示)

首先，扩展FilterInputStream，这是所有InputStream的抽象装饰者。

```
public class LowerCaseInputStream extends FilterInputStream {

    public LowerCaseInputStream(InputStream in) {
        super(in);
    }

    public int read() throws IOException {
        int c = in.read();
        return (c == -1 ? c : Character.toLowerCase((char)c));
    }

    public int read(byte[] b, int offset, int len) throws IOException {
        int result = in.read(b, offset, len);
        for (int i = offset; i < offset+result; i++) {
            b[i] = (byte)Character.toLowerCase((char)b[i]);
        }
        return result;
    }
}
```

记住：我们不在代码列表中提供import和package语句。完整的源代码在<https://wickedlysmart.com/head-first-design-patterns>。

没问题，我只要扩展
FilterInputStream类，并覆
盖read()方法就行了。



现在我们需要实现两个read()方法。它们以字节（或字节数组）为输入，把每个代表大写字母的字节（代表一个字符）转成小写。

测试你的新Java I/O装饰者

快速编写一些代码来测试I/O装饰者：

```
public class InputTest {
    public static void main(String[] args) throws IOException {
        int c;

        try {
            InputStream in =
                new LowerCaseInputStream(
                    new BufferedInputStream(
                        new FileInputStream("test.txt")));

            while((c = in.read()) >= 0) {
                System.out.print((char)c);
            }

            in.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

← 设置FileInputStream并装饰它，首先用BufferedInputStream，然后用崭新的LowerCaseInputStream过滤器。

I know the Decorator Pattern therefore I RULE!

test.txt 文件

↑ 你需要做这个文件。

↑ 只用流来读取字符，一直到文件尾端。一边读一边显示。

运行看看：

```
File Edit Window Help DecoratorsRule
% java InputTest
i know the decorator pattern therefore i rule!
%
```



模式曝光

本周的访谈：
装饰者的坦白

Head First: 欢迎，装饰者模式。听说你最近心情不太好？

装饰者: 是的，我知道大家都把我看成有魅力的设计模式，但是你知道，和每个人一样，我也有自己的黑暗面。

Head First: 愿意和我们分享一些你的麻烦吗？

装饰者: 当然可以。你知道我有能力为设计注入弹性，这是毋庸置疑的，但我也有“黑暗面”。你知道，有时候我会在设计中加入大量的小类，这偶尔会导致设计对其他人来说没那么直白。

Head First: 你能够给我们举个例子吗？

装饰者: 以Java I/O库来说吧。人们第一次接触到这个库时，非常难以理解。但如果他们能认识到这些类都是用来包装InputStream的，一切都会变得简单许多。

Head First: 这听起来没那么糟糕。你还是一个很棒的模式。这个问题只需要对大家做公共教育就可以改善了，对吧？

装饰者: 恐怕不只这些，我还有类型问题。你知道，有些时候，人们在客户代码中依赖特定类型，然后没有经过通盘思考就引入装饰者。现在，我的一个优点是你通常可以透明地插入装饰者，客户甚至不需知道它是在和装饰者打交道。但是，就像我刚说的，有些代码依赖于特定类型，当你开始引入装饰者时，嘭！出状况了！

Head First: 嗯，我想每个人都了解，在插入装饰者时，必须要小心。我不认为这是你的错！

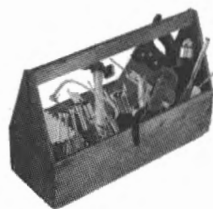
装饰者: 我知道，我也试着不这么想。我还有一个问题，就是引入装饰者会增加实例化组件所需代码的复杂度。一旦用了装饰者，你不只要实例化组件，还要把它包裹进装饰者中，谁知道有几个装饰者。

Head First: 下周我会访谈工厂（Factory）模式和生成器（Builder）模式，我听说他们对解决这个问题有很大的帮助？

装饰者: 确实如此。我应该常和这些家伙聊聊。

Head First: 好吧，我们都认为你是一个很棒的模式，适合用于创建弹性设计，而且遵循开放-关闭原则。开心一点，往好的方面想！

装饰者: 我尽量，谢谢你。



你的设计工具箱中的工具

又搞定了一章，工具箱里有了新的原则和模式。

OO基础

OO原则

封装变化。

组合优于继承。

针对接口编程，而不是针对实现编程。

为交互对象之间的松耦合设计而努力。

类应该对扩展开放，对修改关闭。

对象
装
态
承

现在有了开放-关闭原则引导我们。我们会努力地设计系统，好让关闭的部分和新扩展的部分隔离。

OO模式

装饰者，给对象动态附加额外的责任。对于扩展功能，除了子类化之外，装饰者提供了弹性的替代做法。

这是我们的第一个创建符合开放-关闭原则的设计的模式。真的是第一个吗？有没有其他我们用过的模式也遵循此原则？

要点

- 继承是扩展形式之一，但未必是达到弹性设计的最佳方式。
- 在我们的设计中，应该允许行为可以被扩展，而无需修改已有代码。
- 组合和委托经常可以用来在运行时添加新行为。
- 装饰者模式提供了子类化扩展行为的替代品。
- 装饰者模式涉及一群装饰者类，这些类用来包装具体组件。
- 装饰者类反映了它们所装饰的组件类型（事实上，它们和所装饰的组件类型相同，都经过了继承或接口实现）。
- 装饰者通过在对组件的方法调用之前（或/和之后，甚至在那一刻）添加功能改变其组件的行为。
- 你可以用任意数目的装饰者来包裹一个组件。
- 装饰者一般对组件的客户是透明的，除非客户依赖于组件的具体类型。
- 装饰者会导致设计中出现许多小对象，过度使用会让代码变得复杂。



削尖你的铅笔 答案

为以下类编写cost()方法（用伪Java就可以）。
答案如下：

```
public class Beverage {

    // 声明实例变量milkCost,
    // soyCost, mochaCost和whipCost, 以及
    // milk, soy, mocha和whip的
    // getters和setters。

    public double cost() {

        double condimentCost = 0.0;
        if (hasMilk()) {
            condimentCost += milkCost;
        }
        if (hasSoy()) {
            condimentCost += soyCost;
        }
        if (hasMocha()) {
            condimentCost += mochaCost;
        }
        if (hasWhip()) {
            condimentCost += whipCost;
        }
        return condimentCost;
    }
}

public class DarkRoast extends Beverage {

    public DarkRoast() {
        description = "Most Excellent Dark Roast";
    }

    public double cost() {
        return 1.99 + super.cost();
    }
}
```

削尖你的铅笔

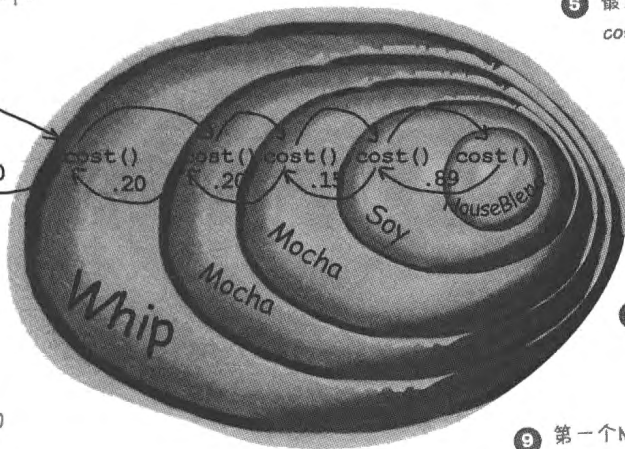
答案

新咖啡师训练

“双倍摩卡豆奶拿铁带奶泡”



- 1 首先，我们调用最外层装饰者Whip。
- 2 Whip调用Mocha上的cost()。
- 3 Mocha调用另一个Mocha上的cost()。
- 4 接下来，Mocha调用Soy上的cost()。
- 5 最后的调料！Soy调用HouseBlend的cost()。
- 6 HouseBlend的cost()方法返回.89给Soy后，出栈。
- 7 Soy的cost()方法加上.15，返回结果，然后出栈。
- 8 第二个Mocha的cost()方法加上.20，返回结果，然后出栈。
- 9 第一个Mocha的cost()方法加上.20，返回结果，然后出栈。
- 10 最后，结果返回到Whip的cost()，该方法添加.10，得到最终价钱\$1.54。





削尖你的铅笔 答案

我们在星巴兹的朋友在菜单中引进了容量。现在你可以购买小杯 (tall)、中杯 (grande)、大杯 (venti)。星巴兹认为这是咖啡行业必备的，因此他们给Beverage类添加了两方法：setSize()和getSize()。他们也希望调料根据容量收费，例如：小、中、大容量的咖啡加上豆奶，分别加收10美分、15美分、20美分。更新后的Beverage类如下。

怎样修改装饰者类来处理这样的需求变更？

答案如下。

```
public abstract class CondimentDecorator extends Beverage {
    public Beverage beverage;
    public abstract String getDescription();

    public Size getSize() {
        return beverage.getSize();
    }
}
```

我们为装饰者添加一个方法getSize()，简单返回饮料的容量。

```
public class Soy extends CondimentDecorator {
    public Soy(Beverage beverage) {
        this.beverage = beverage;
    }

    public String getDescription() {
        return beverage.getDescription() + ", Soy";
    }
}
```

```
public double cost() {
    double cost = beverage.cost();
    if (beverage.getSize() == Size.TALL) {
        cost += .10;
    } else if (beverage.getSize() == Size.GRANDE) {
        cost += .15;
    } else if (beverage.getSize() == Size.VENTI) {
        cost += .20;
    }
    return cost;
}
```

在这里我们取得容量（一直扩散到具体的饮料），然后加上适当的价格。